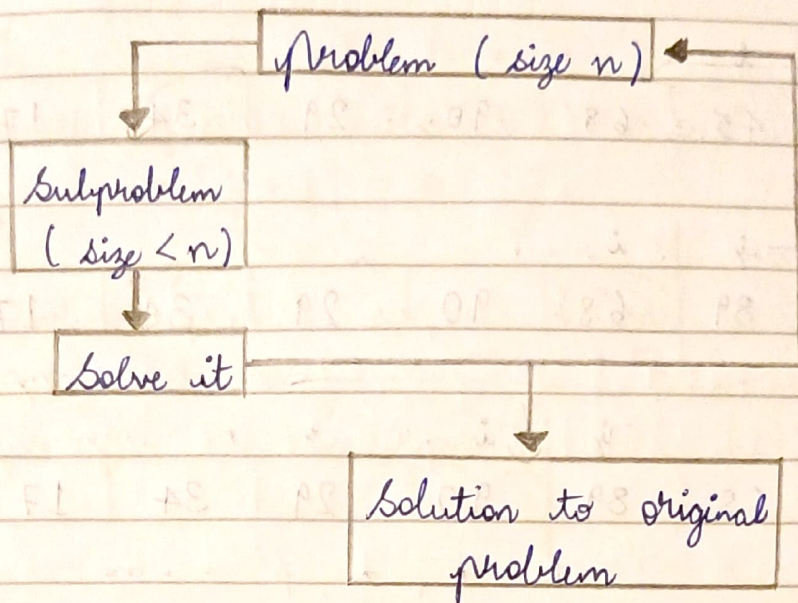


26/6 Decrease & Conquer →



- Breaking down complex problem into smaller instance, solving them & combining solution

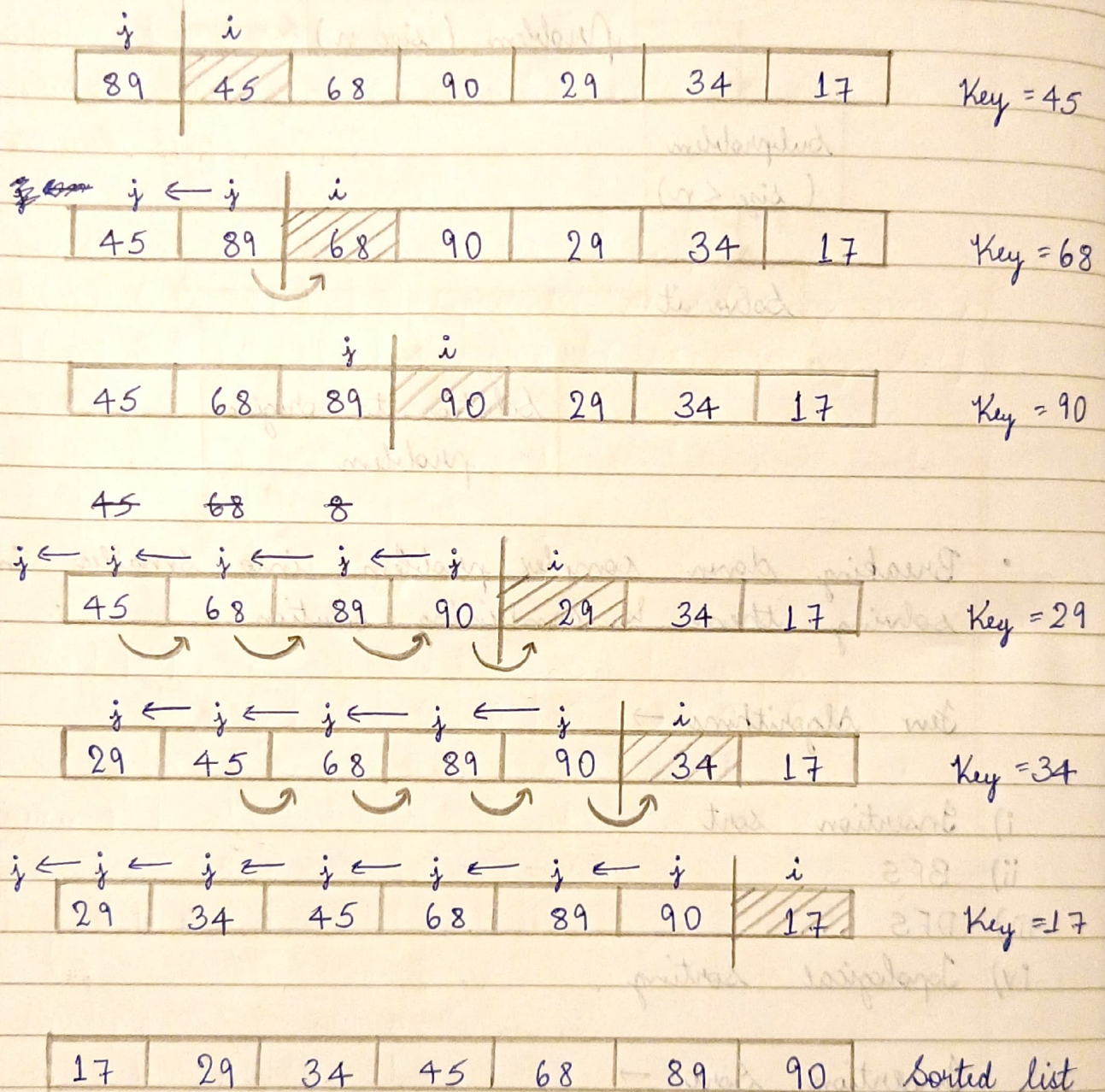
Few Algorithms →

- i) Insertion sort
- ii) BFS
- iii) DFS
- iv) Topological sorting

Insertion Sort →

- Sorting algo
- Based on decrease & conquer technique
- $T(n) \in O(n^2)$
- $T(n) \in \Theta(n^2)$
- $T(n) \in \Omega(n)$
- Performs better in partially sorted list

eg 89, 45, 68, 90, 29, 34, 17



ALGORITHM InsertionSort ($a[0 \dots n-1]$) $\rightarrow$

Purpose Sort given array using insertion sort method

Input $a[0 \dots n-1] \Rightarrow$ Array

Output $a[0 \dots n-1] \Rightarrow$ Sorted array

Algo. for $i = 1$ to $n-1$
 $j = i - 1$
 $key = a[i]$
 while ($j \geq 0$ and $a[j] > key$)
 $a[j+1] = a[j]$
 $j --$
 endwhile
 $a[j+1] = key$
 endfor

Analysis $\rightarrow$

- Input size $\rightarrow n$
- Basic operation $\rightarrow$ key comparison ($j \geq 0$ and $a[j] > key$)

$$\begin{aligned}
 T(n) &= \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} (i - 1 - 0 + 1) = \sum_{i=1}^{n-1} i \\
 &= 1 + 2 + 3 + \dots + n-1 = \frac{(n-1)(n-1+1)}{2} = \frac{n(n-1)}{2} \approx n^2
 \end{aligned}$$

$$\therefore T(n) \in O(n^2), \theta(n^2)$$

Note Best case $\rightarrow$ If given list is already sorted

$$T(n) = \sum_{i=0}^{n-1} 1 = (n-1 - 0 + 1) = n$$

$$\therefore T(n) \in \Omega(n)$$

27/6 Graph Traversal $\rightarrow$

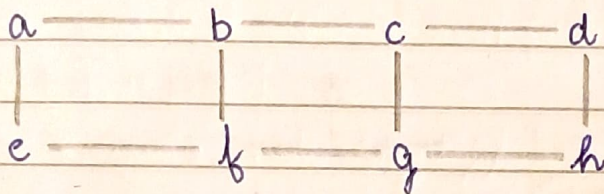
i) BFS

ii) DFS

Breadth First Search $\rightarrow$

- Fundamental graph traversal algo.
- Involves visiting all connected nodes of graph in level-by-level manner
- Uses queue DS
- Based on decrease & conquer technique
- $T(n) \in \Theta(|V|^2)$ [if represented as adjacency matrix]
- $T(n) \in \Theta(|V| + |E|)$ [if represented as adjacency list]

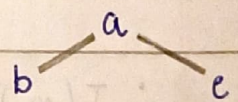
eg



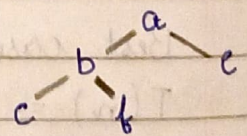
visited []	a	b	c	d	e	f	g	h
	0	0	0	0	0	0	0	0

queue []

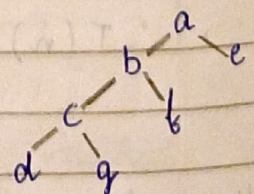
queue []	a	b	e					
	f							



queue []	a	b	e	c	f			
		f						

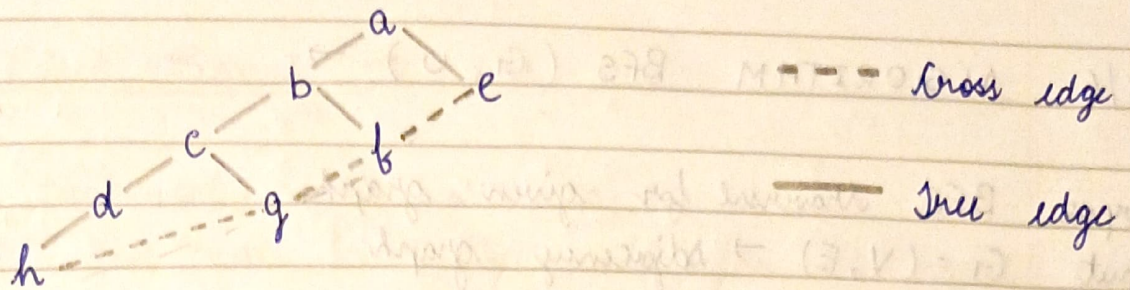


queue []	a	b	e	c	f	d	g	
			f $\rightarrow$ f					



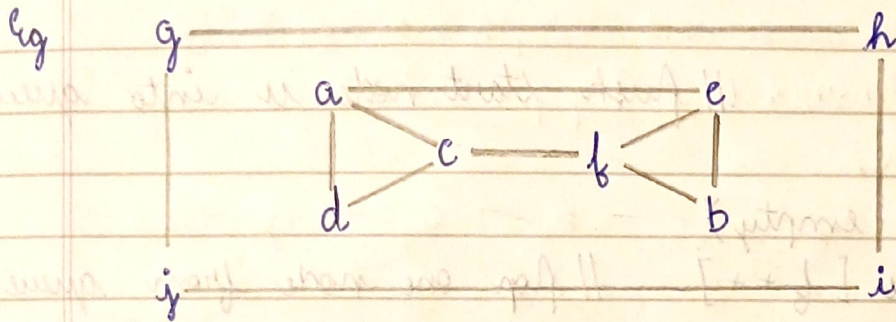
queue []	a	b	e	c	f	d	g	h
					f $\rightarrow$ f			

Note front f should be visited



BFS order of traversal $\rightarrow$

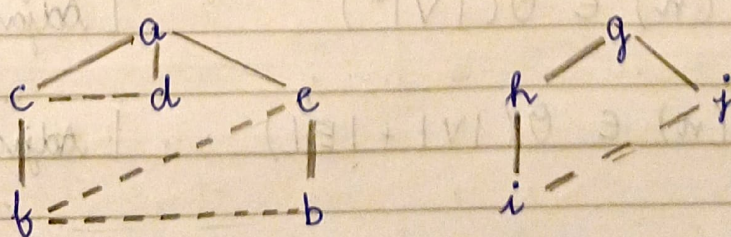
$a_1 \rightarrow b_2 \rightarrow c_3 \rightarrow d_4 \rightarrow f_5 \rightarrow g_7 \rightarrow h_8$



queue	a	c	d	e						
queue	a	c	d	e	f					
queue	a	c	d	e	f	b				

Separate $\rightarrow$... g h j i

All nodes are not visited $\Rightarrow$ Not connected graph [separate BFS tree]



BFS order of traversal $\rightarrow$

$a_1 \rightarrow c_2 \rightarrow d_3 \rightarrow e_4 \rightarrow f_5 \rightarrow b_6 \rightarrow g_7 \rightarrow h_8 \rightarrow j_9 \rightarrow i_{10}$

28/6 ALGORITHM BFS (G, u) $\rightarrow$

Purpose BFS traversal for given graph

Input $G = (V, E) \rightarrow$ Adjacency graph
 $u \rightarrow$ start vertex

Output Order of BFS traversal
 Algo.

```

for  $i = 0$  to  $|V| - 1$ 
     $visited[i] = 0$  // Mark all nodes as unvisited
count = 0
queue[++n] = u // push start node u into queue
visited[u] = 1
while (queue not empty)
     $u = queue[f++]$  // pop one node from queue
    count = count + 1
    display(a)
    for  $v = 0$  to  $|V| - 1$  // Each vertex v from graph
        if ( $visited[v] == 0$  and  $graph[u][v] == 1$ )
            queue[++n] = v
            visited[v] = 1
  
```

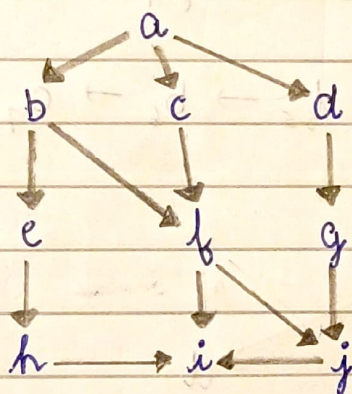
Analysis $\rightarrow$

- $\therefore T(n) \in \Theta(|V|^2)$ [Adjacency matrix]
- $\therefore T(n) \in \Theta(|V| + |E|)$ [Adjacency list]

Depth First Search →

- Graph traversal / searching algo.
- Starts at root node & explores as far as possible along each branch before backtracking
- Uses stack DS
- Uses extra memory to keep track of nodes
- Based on decrease & conquer technique
- $T(n) \in O(|V|^2)$ [if represented as adjacency matrix]
- $T(n) \in O(|V| + |E|)$ [if represented as adjacency list]

eg



visited []	a	b	c	d	e	f	g	h	i	j
	0	0	0	0	0	0	0	0	0	0

Stack

										i
										h
										e
										b
										a
	push(a)		a → b		b → e		e → h		h → i	
	start node		push(b)		push(e)		push(h)		push(i)	
		a		b		e		h		i
			a			b		e		h
						a		b		e
								a		b
										a

	h	no edge		no edge		b → b		f → j		j	no edge		f
dead end	e		e				f		f				b
	b		b		b		b		b				b
pop(i)	a	pop(h)	a	pop(e)	a	push(f)	a	push(j)	a	pop(j)			a

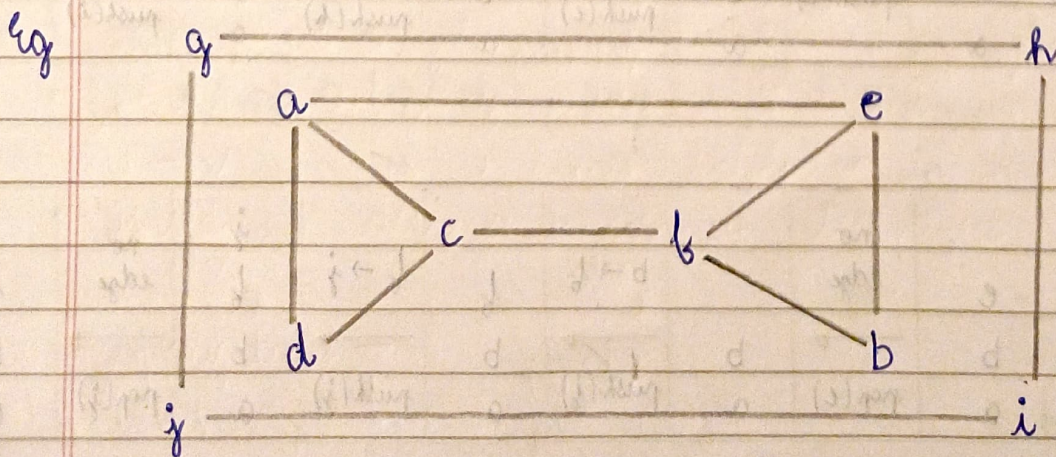
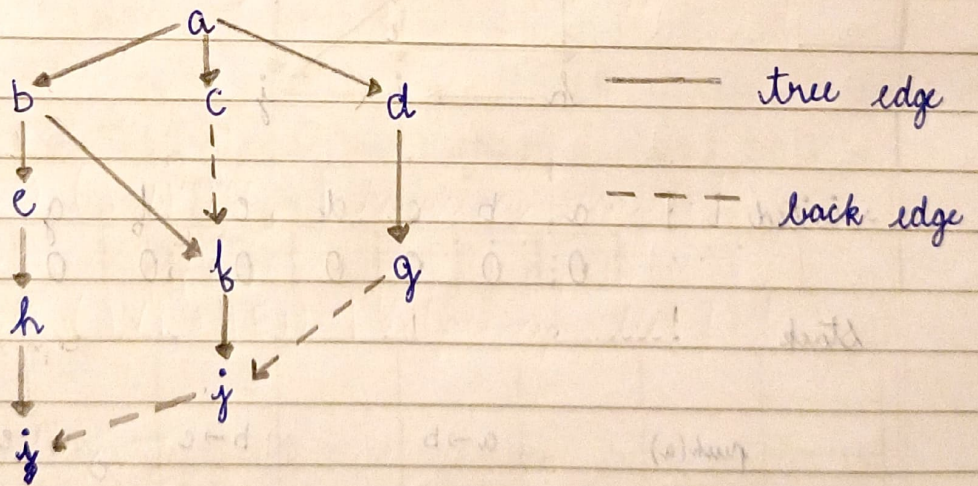
no edge		no edge		a → c		no edge		a → d	
→	b	→	a	→	c	→	a	→	d
pop(b)	a	pop(b)	a	push(c)	a	pop(c)	a	push(d)	d
					a				a

d → g	g	no edge		no edge		no edge		Empty stack	
→	d	→	d	→		→			
push(g)	a	pop(g)	a	pop(d)	a	pop(a)			

DFS order of traversal → (pop order)

$i_1 \rightarrow h_2 \rightarrow e_3 \rightarrow j_4 \rightarrow f_5 \rightarrow b_6 \rightarrow c_7 \rightarrow g_8 \rightarrow d_9 \rightarrow a_{10}$

DFS tree →



	a	b	c	d	e	f	g	h	i	j
visited[]	0	0	0	0	0	0	0	0	0	0

	start node a		a → c		c → d		d		no edge	
Empty stack	→		→		→		→		→	
stack	push(a)	a	push(c)	c a	push(d)	d c a	pop(d)	c a		

			e							
c → f	f	f → b	b	b → e	b	no edge	b	no edge		
→	c	→	f	→	f	→	f	→	f	
push(f)	a	push(b)	a	push(e)	a	pop(e)	a	pop(b)	a	

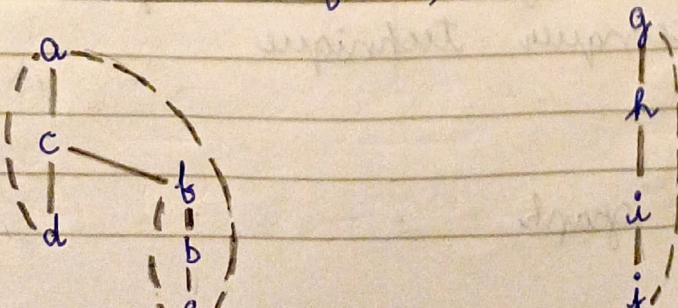
					unvisited vertex		g → h			
no edge		no edge		no edge	→		→			
pop(f)	c	pop(c)	a	pop(a)	push(g)	g	push(h)	h g		

h → i	i	i → j	j	no edge	i	no edge		no edge		no edge
→	h	→	h	→	h	→	h	→		→
push(i)	g	push(j)	g	pop(j)	j	pop(i)	j	pop(h)	j	pop(j)

DFS order of traversal (stack pop order) →

$d_1 \rightarrow e_2 \rightarrow b_3 \rightarrow f_4 \rightarrow c_5 \rightarrow a_6 \rightarrow j_7 \rightarrow i_8 \rightarrow h_9 \rightarrow g_{10}$

DFS tree (or forest, in this case) →



(u)

ALGORITHM DFS (~~G~~, u) →

Purpose DFS traversal for given graph

Input $G = (V, E) \rightarrow$ Adjacency graph
 $u \rightarrow$ start vertex

Output Order of DFS traversal
Algo.

count = count + 1

for $v = 0$ to $|V| - 1$

if (visited $[v] == 0$ and graph $[u][v] == 1$)

visited $[v] = 1$

DFS (v)

endif

endfor

display (u)

Analysis →

- $\therefore T(n) \in \Theta(|V|^2)$ [Adjacency matrix]
- $\therefore T(n) \in \Theta(|V| + |E|)$ [Adjacency list]

2/7 Topological Sorting →

- For direct acyclic graph
- Linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v in ordering
- Based on decrease & conquer technique

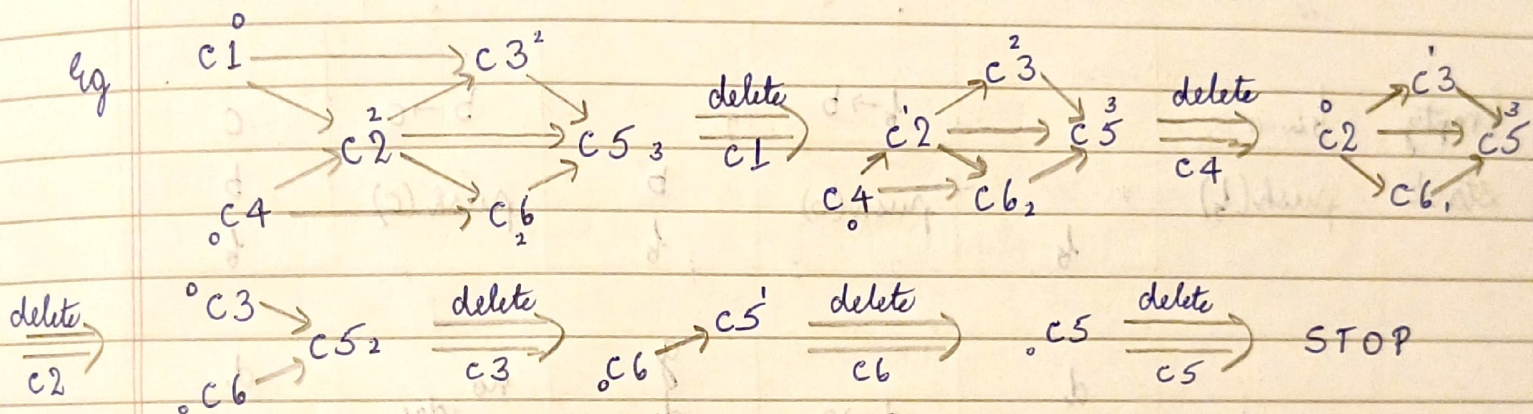
Applications →

- Detect cycle in acyclic graph

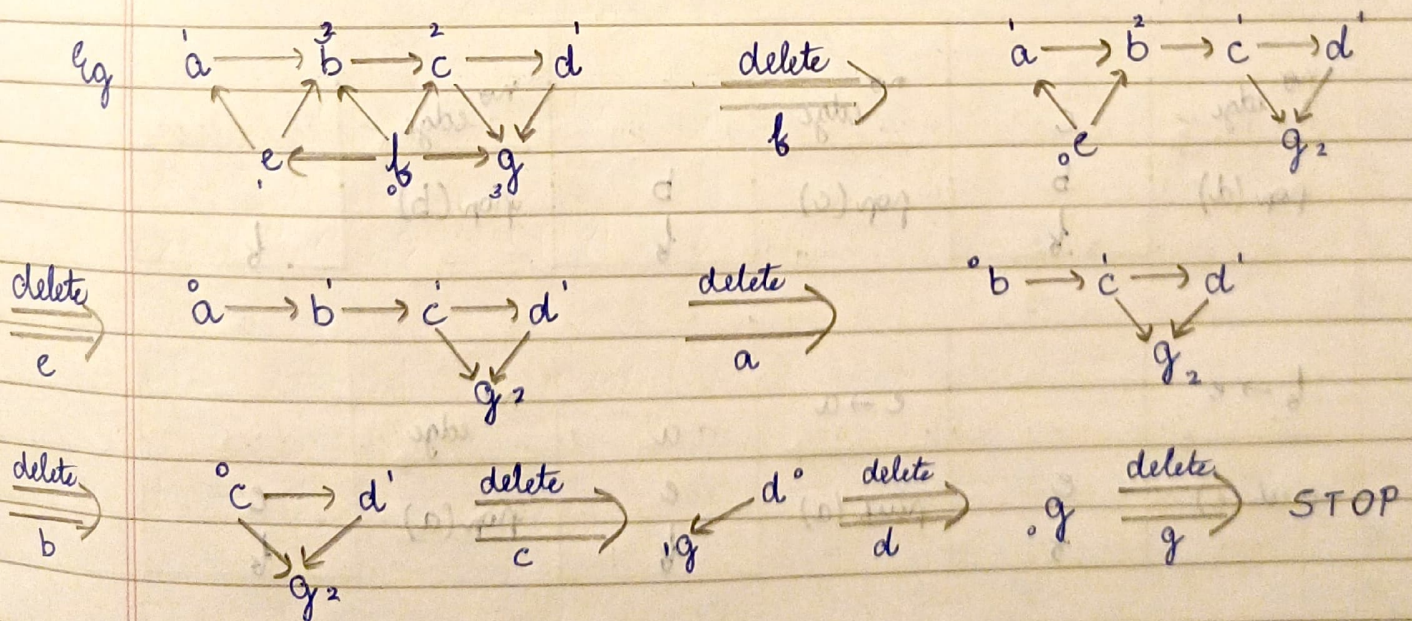
- Helps in scheduling task or event based dependencies
- Deadlock detection in OS

Can be solved by 2 methods:

i) Source Removal → Delete node with indegree 0

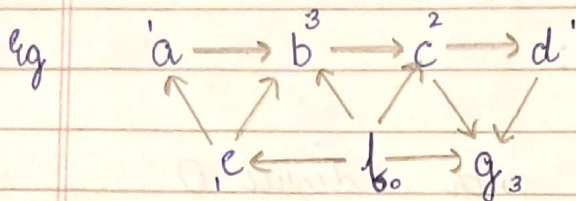


∴ TS Order : $c1 \rightarrow c4 \rightarrow c2 \rightarrow c3 \rightarrow c6 \rightarrow c5$



TS Order :

$f \rightarrow e \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow g$

ii) DFS →empty
stackb_{in} = 0

push(b)

b

b → b

push(b)

b

b

b → c

push(c)

c

b

b

c → d

push(d)

d

c

b

b

d → g

push(g)

g

d

c

b

b

no
edge

pop(g)

d

c

b

b

no
edge

pop(d)

c

b

b

no
edge

pop(c)

b

b

no
edge

pop(b)

b

b → e

push(e)

e

b

e → a

push(a)

a

e

b

no
edge

pop(a)

c

b

no
edge

pop(e)

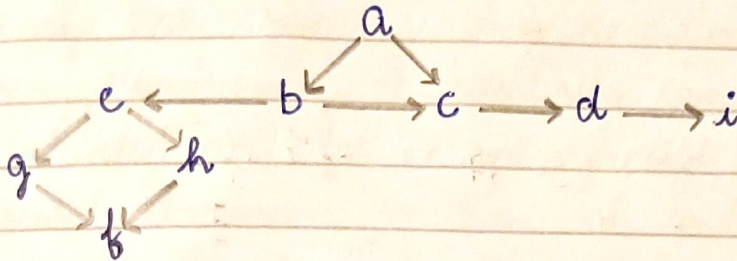
b

no
edge

pop(b)

$\therefore$ pop Order : $g \rightarrow d \rightarrow c \rightarrow b \rightarrow a \rightarrow e \rightarrow f$
 $\therefore$ DFS Order : $f \rightarrow e \rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow g$

eg



Empty stack	$a_{in=0}$ $\rightarrow$ push(a)	a	$a \rightarrow b$ $\rightarrow$ push(b)	b a	$b \rightarrow e$ $\rightarrow$ push(e)	e b a	$e \rightarrow g$ $\rightarrow$ push(g)	g e b a
	$g \rightarrow f$ $\rightarrow$ push(f)	f g e b a	no edge $\rightarrow$ pop(f)	g e b a	no edge $\rightarrow$ pop(g)	e b a	$e \rightarrow h$ $\rightarrow$ push(h)	h e b a
	no edge $\rightarrow$ pop(h)	e b a	no edge $\rightarrow$ pop(e)	b a	$b \rightarrow c$ $\rightarrow$ push(c)	c b a	$c \rightarrow d$ $\rightarrow$ push(d)	d c b a
	$d \rightarrow i$ $\rightarrow$ push(i)	i d c b a	no edge $\rightarrow$ pop(i)	d c b a	no edge $\rightarrow$ pop(d)	c b a	no edge $\rightarrow$ pop(c)	b a
	no edge $\rightarrow$ pop(b)	a	no edge $\rightarrow$ pop(a)					

pop Order :

 $f \rightarrow g \rightarrow h \rightarrow e \rightarrow i \rightarrow d \rightarrow c \rightarrow b \rightarrow a$

DFS Order :

 $a \rightarrow b \rightarrow c \rightarrow d \rightarrow i \rightarrow e \rightarrow h \rightarrow g \rightarrow f$

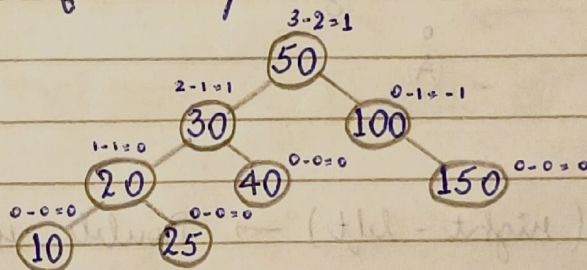
4/7 For BST, $T(n) \in O(\log_2 n)$
 Drawback of BST $\rightarrow T(n) \in O(n)$ for skewed BST
 To overcome this, we use Balanced Search Tree

Balanced Search Tree $\rightarrow$

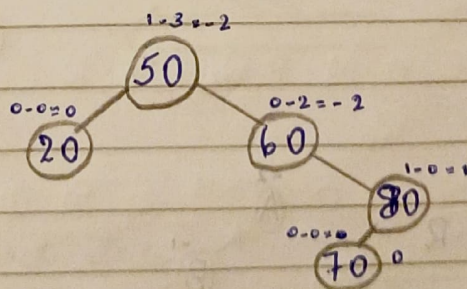
- i) AVL tree ii) 2-3 tree iii) B-tree

AVL Tree $\rightarrow$

- Named after Adelson-Velsky & Landis
- Self balancing BST
- Balance factor = Height of left subtree - Height of right subtree
- Balance factor of every node should be 0 or -1 or +1



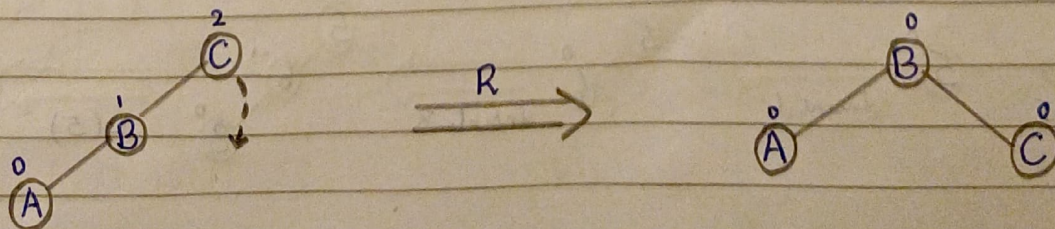
$\therefore$ It is AVL tree



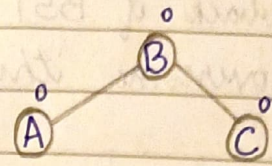
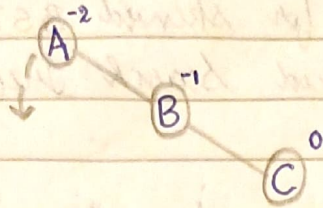
$\therefore$ Not AVL tree

AVL Rotation $\rightarrow$

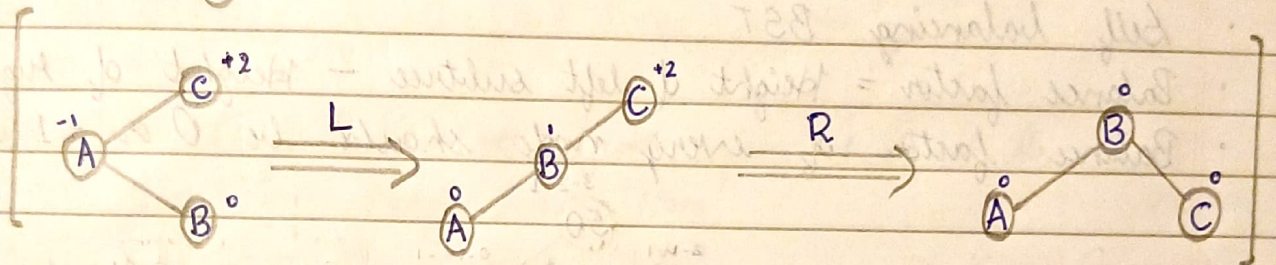
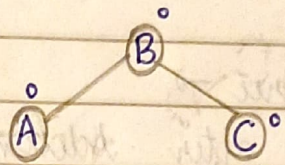
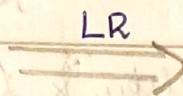
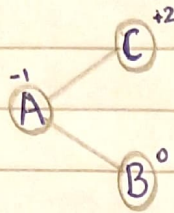
- i) R-rotation (right) $\rightarrow$ single rotation



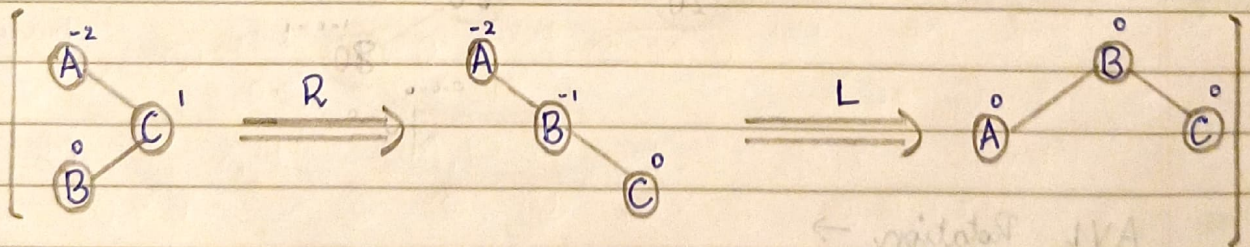
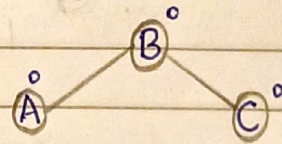
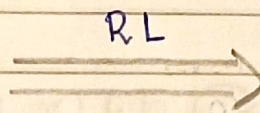
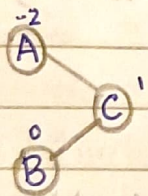
ii) L - notation (left) $\rightarrow$ Single rotation



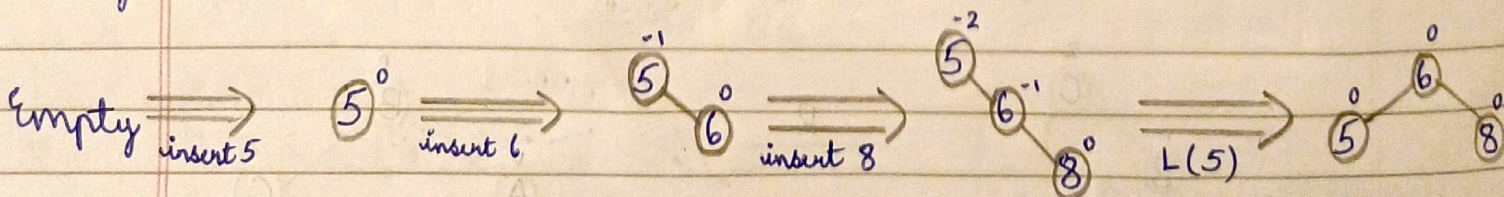
iii) LR - notation (left-right) $\rightarrow$ Double rotation

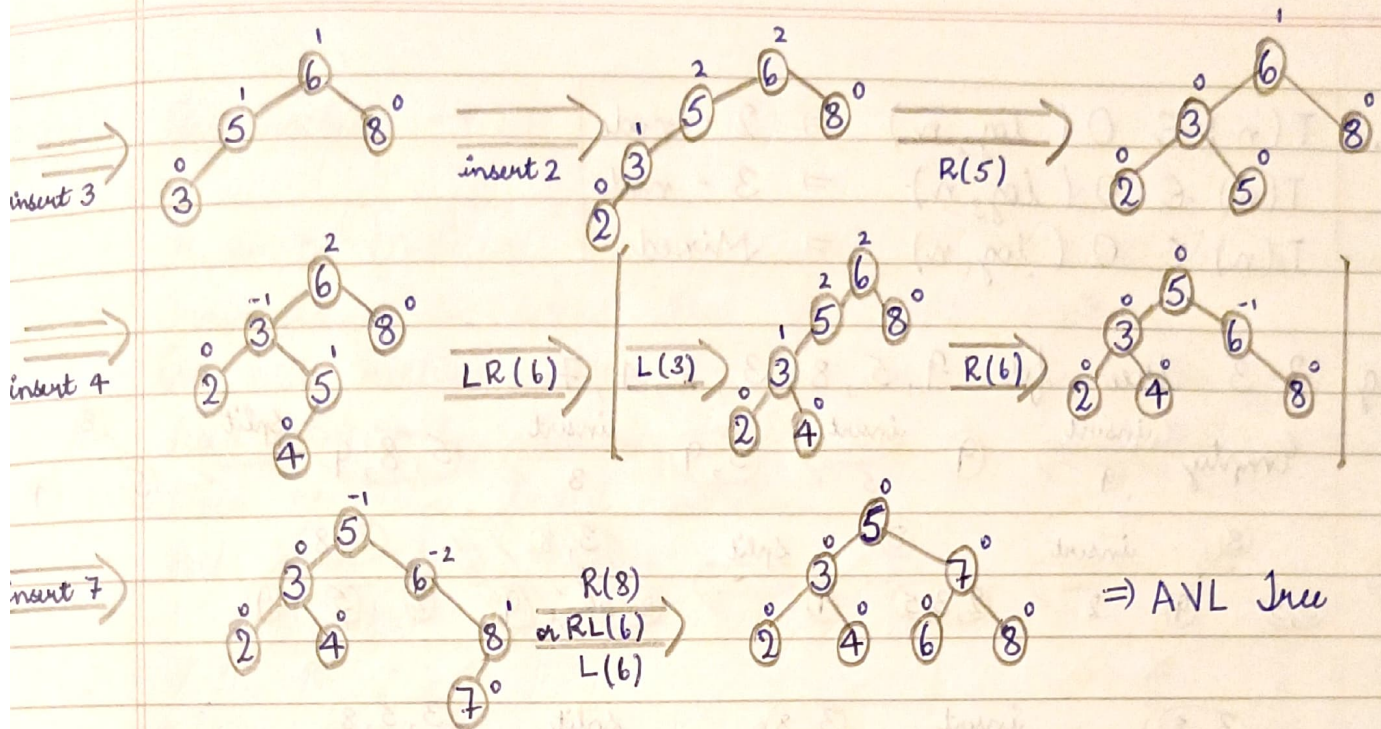


iv) RL - notation (right-left) $\rightarrow$ Double rotation

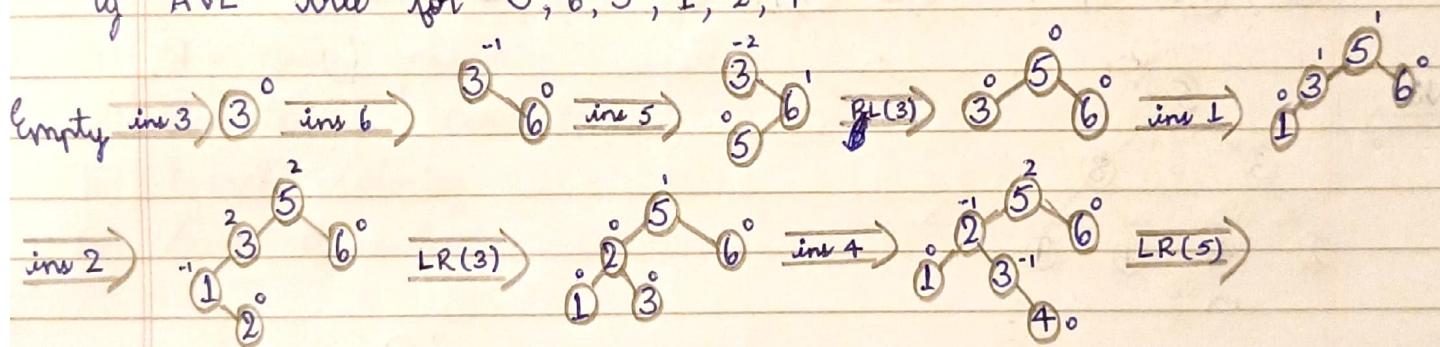


eg AVL tree for 5, 6, 8, 3, 2, 4, 7





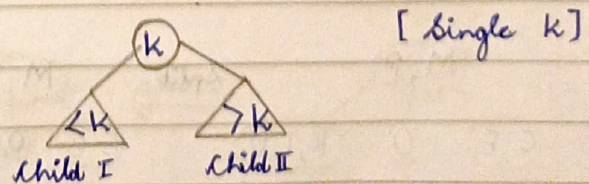
eg AVL tree for 3, 6, 5, 1, 2, 4



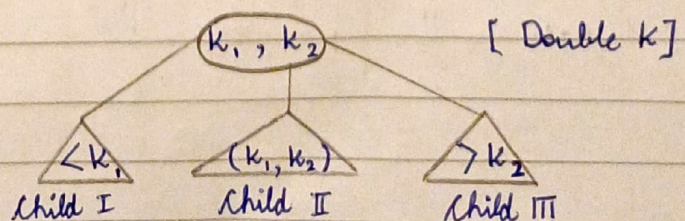
11/7 2 - 3 Trees $\rightarrow$

- Self balancing search tree
- Tree that can have nodes of two kinds - 2-nodes & 3-nodes

2-nodes $\rightarrow$ Two children

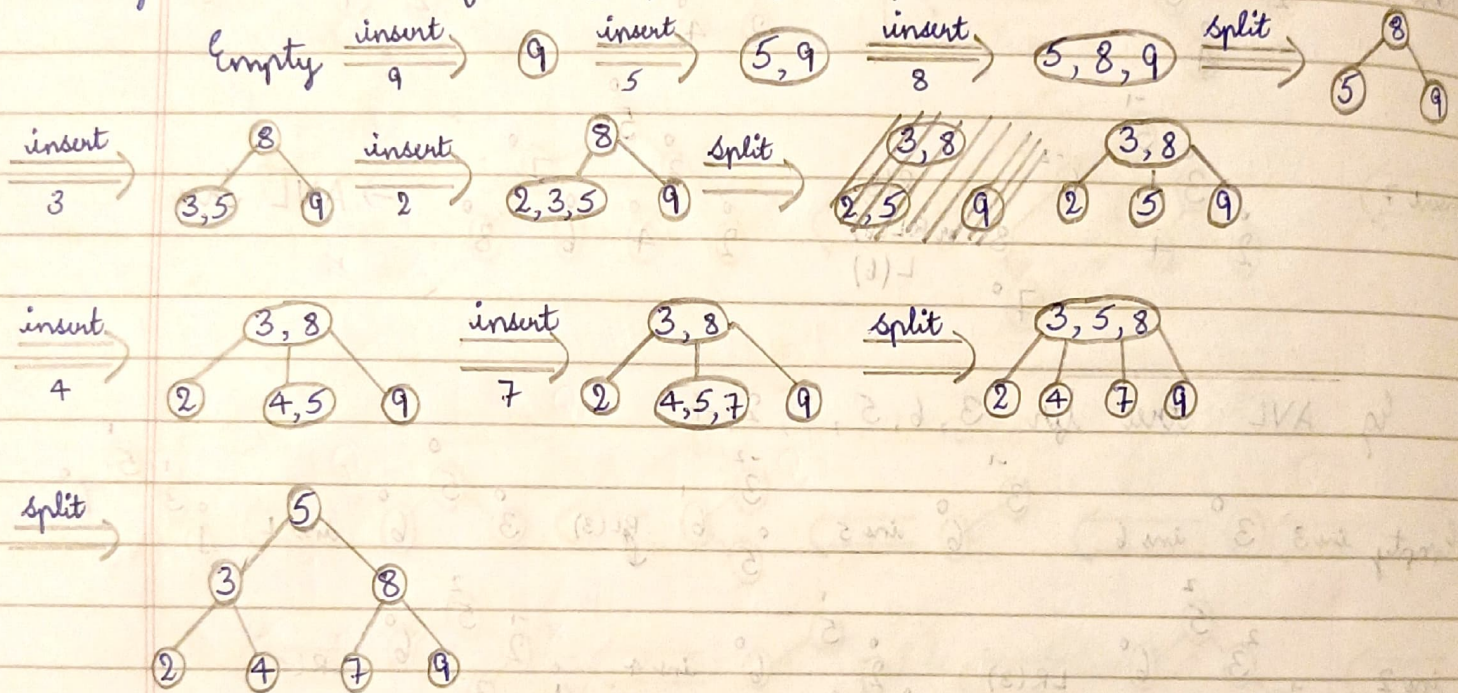


3-nodes $\rightarrow$ Three children



$$\therefore \begin{array}{l} T(n) \in O(\log_2 n) \Rightarrow 2\text{-node} \\ T(n) \in O(\log_3 n) \Rightarrow 3\text{-node} \\ T(n) \in O(\log_e n) \Rightarrow \text{Mixed} \end{array}$$

eg 2-3 tree for 9, 5, 8, 3, 2, 4, 7



eg 2-3 tree for C, O, M, P, U, T, E, R

